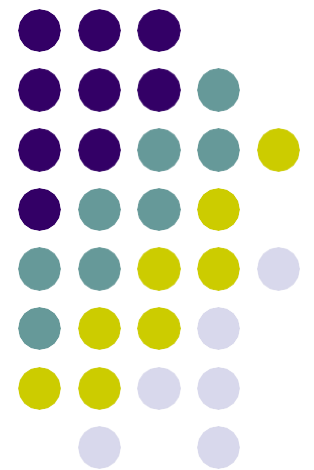


Digital Image Processing

Lecture 3: Filters

Prof Emmanuel Agu

Computer Science Dept.
Worcester Polytechnic Institute (WPI)





What Is Image Enhancement?

Image enhancement makes images more useful by:

- Highlighting interesting detail in images
- Removing noise from images
- Making images more visually appealing



Image Enhancement Examples (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

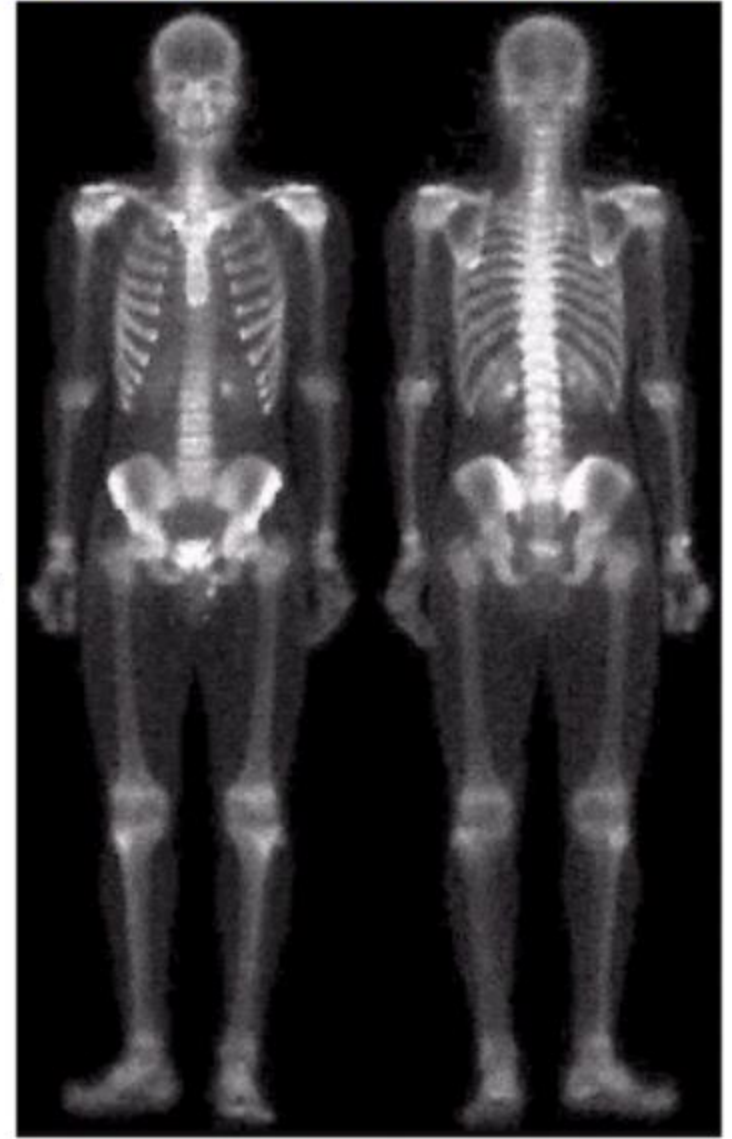
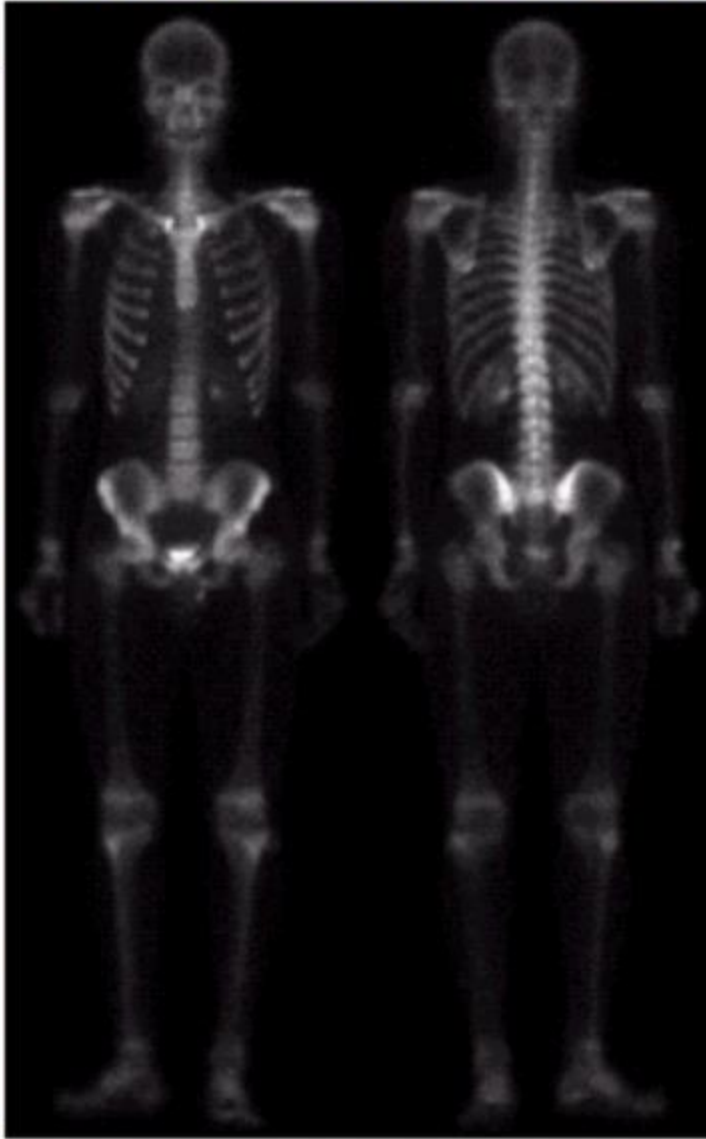
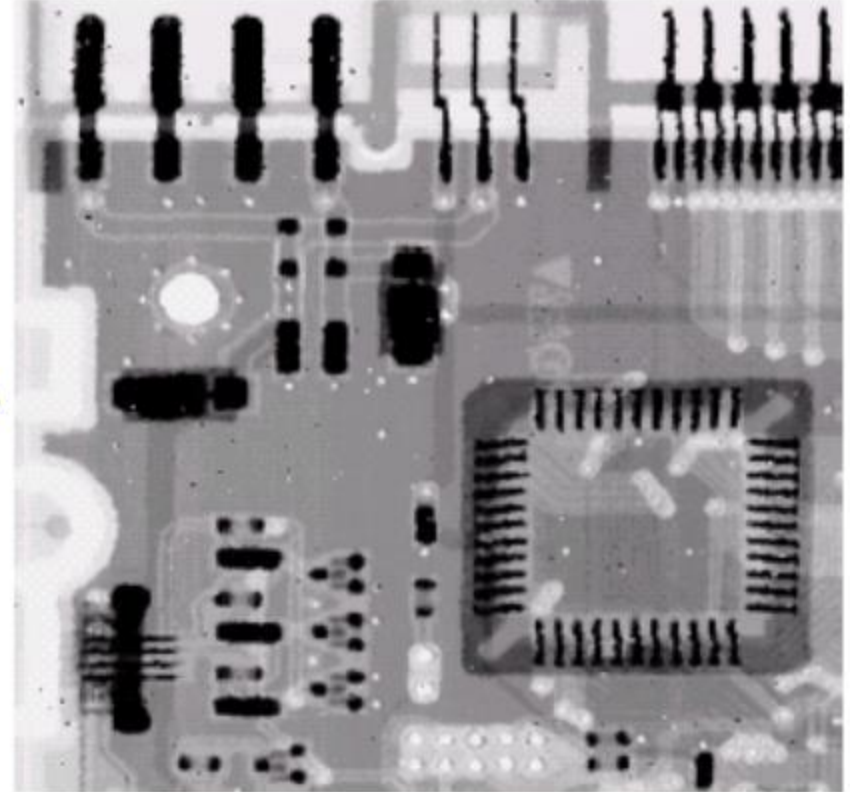
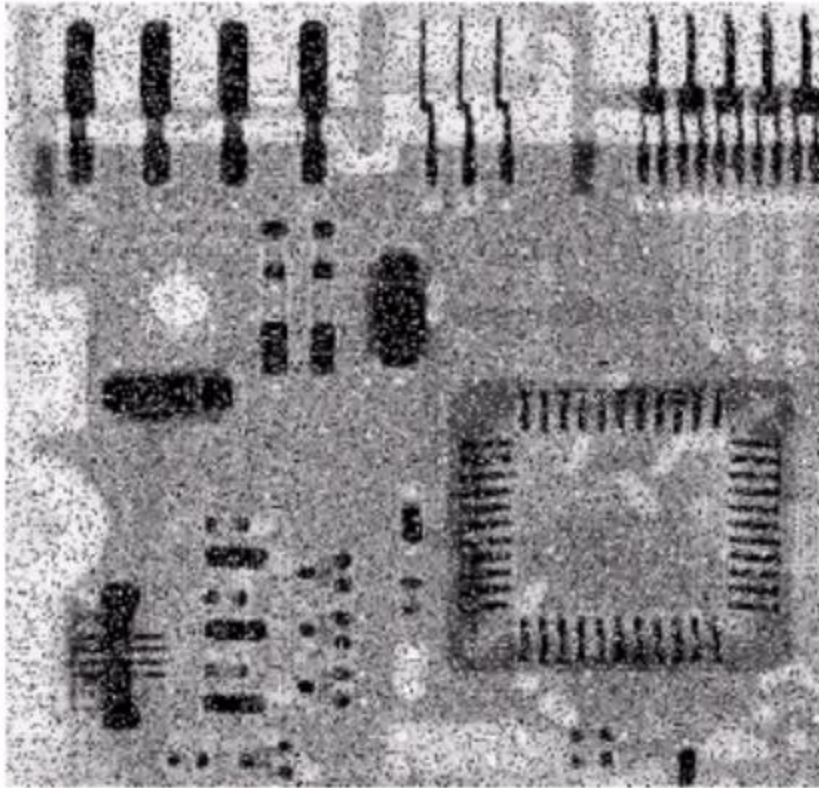


Image Enhancement Examples (cont...)



Images taken from Gonzalez & Woods, Digital Image Processing (2002)





Spatial & Frequency Domains

There are two broad categories of image enhancement techniques

- Spatial domain techniques
 - Direct manipulation of image pixels (intensity values)
- Frequency domain techniques
 - Manipulation of Fourier transform or wavelet transform of an image

What is a Filter?



- Capabilities of point operations are limited
- **Filters:** combine **pixel's value** + **values of neighbors**
- **E.g blurring:** Compute average intensity of block of pixels

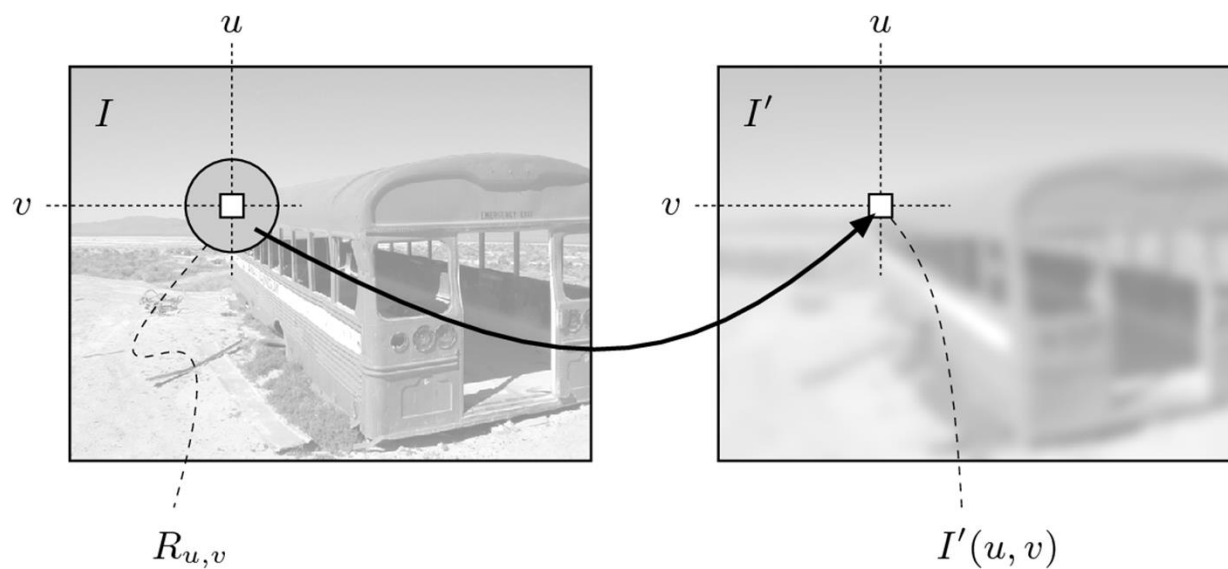


- Combining multiple pixels needed for certain operations:
 - Blurring, Smoothing
 - Sharpening

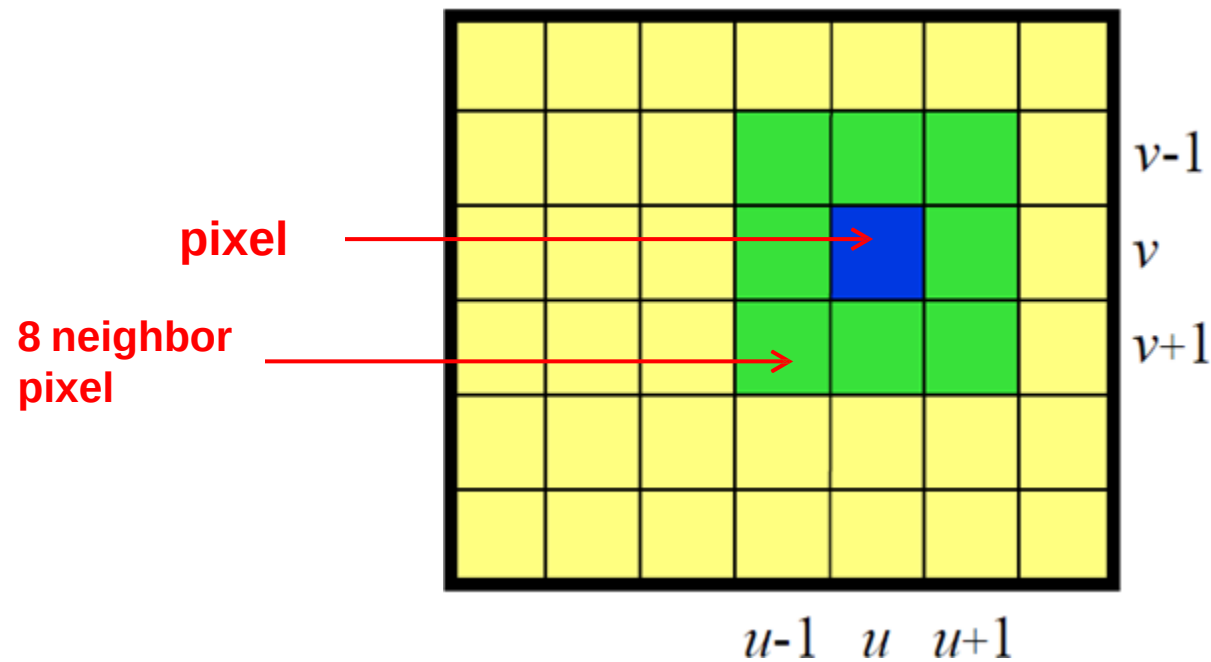


Definition: Spatial Filter

- An image operation that combines each pixel's intensity $I(u, v)$ with that of neighboring pixels
- **E.g:** average/weighted average of group of pixels



Example: Average (Mean) of 3x3 Neighborhood



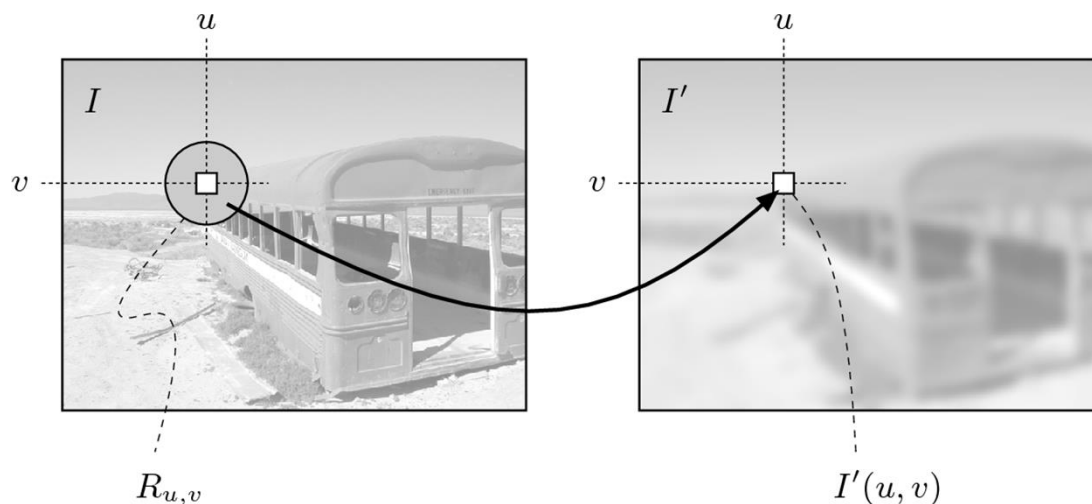
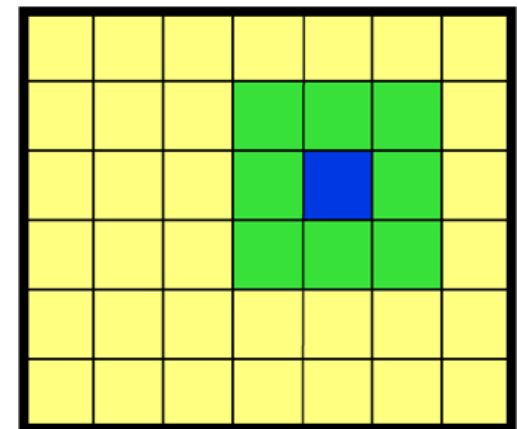
Blurring: Replace each pixel with AVERAGE Intensity of pixel + neighbors



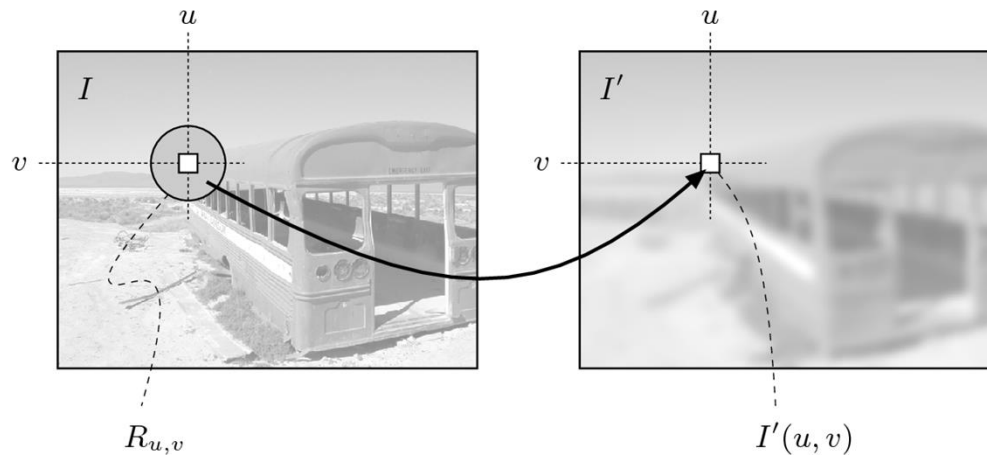
Smoothing an Image by Averaging

- Replace each pixel by average of pixel + neighbors
- For 3x3 neighborhood:

$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$



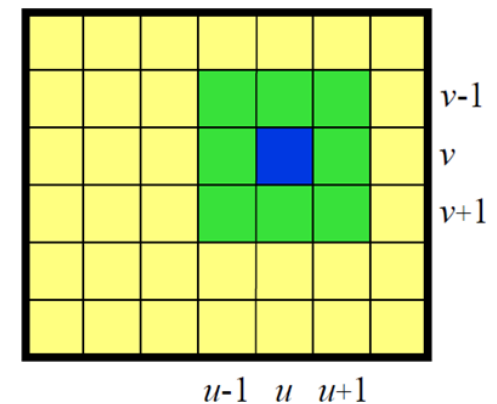
Smoothing an Image by Averaging



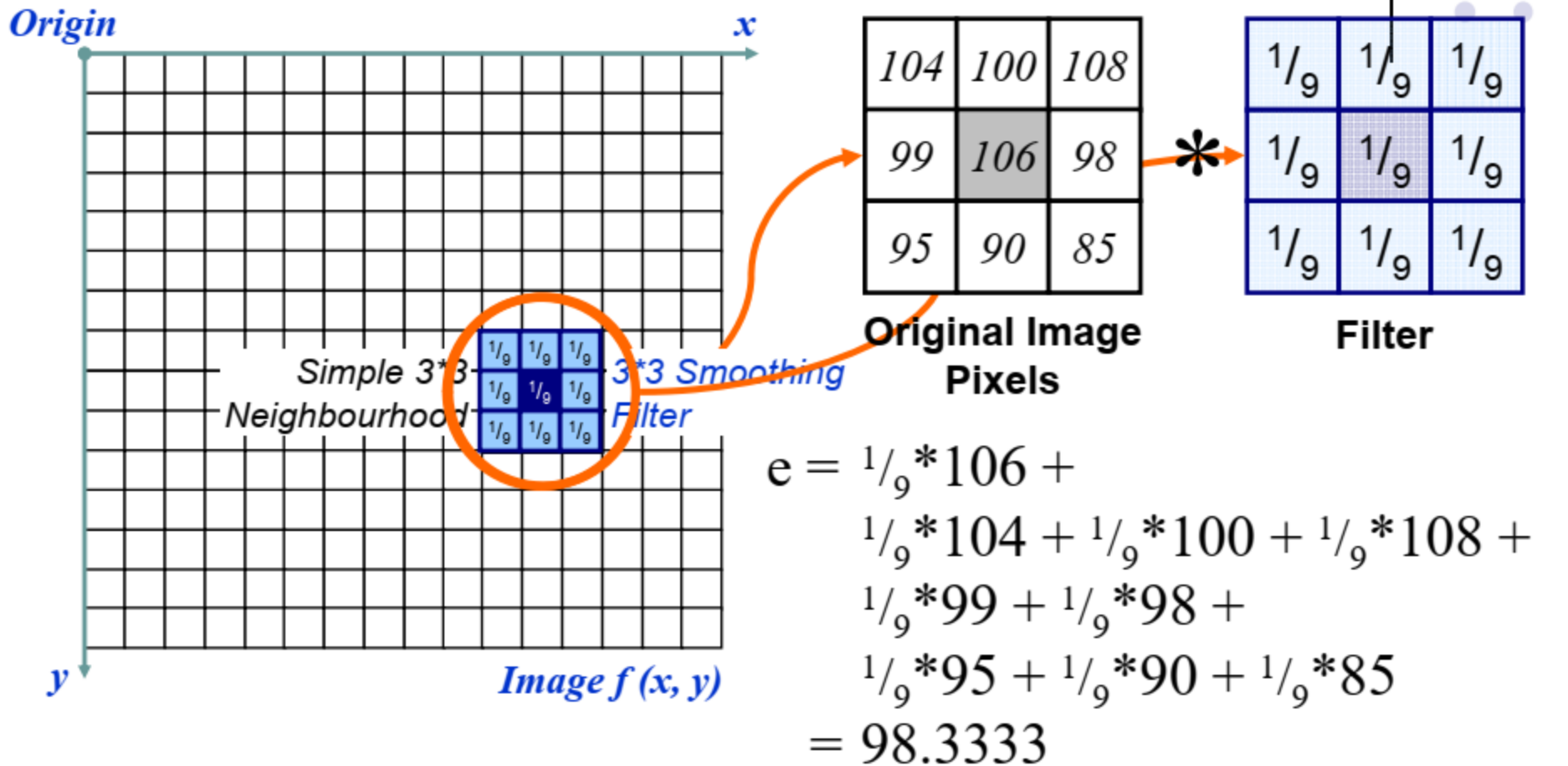
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

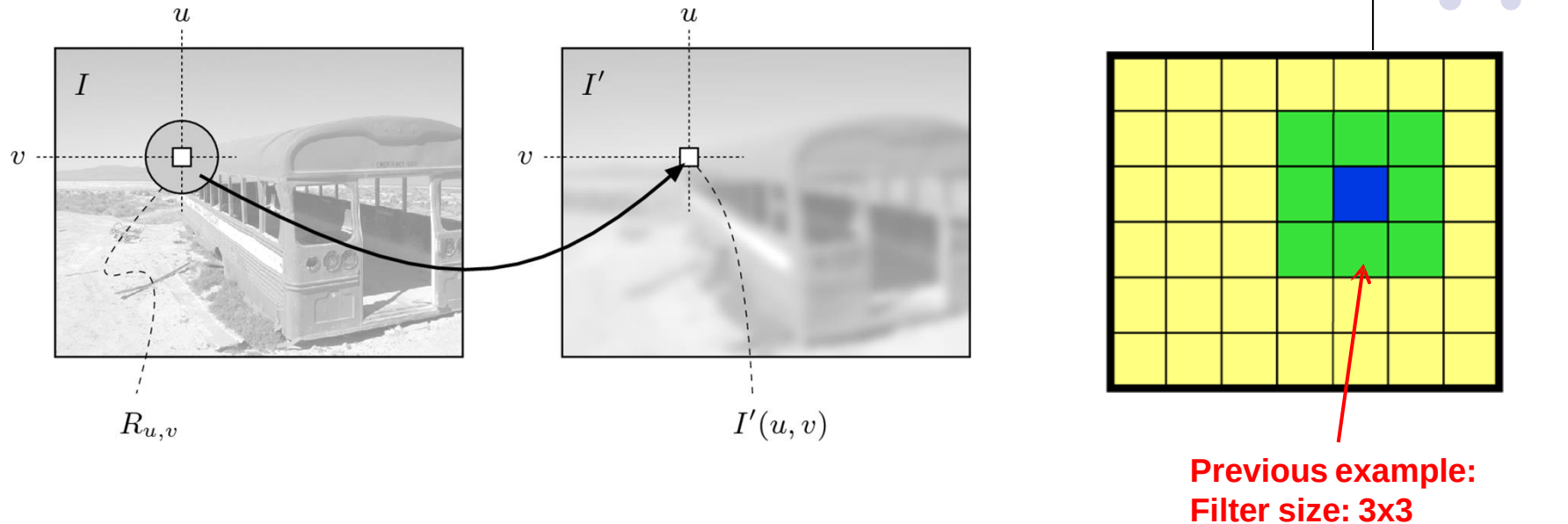


Example: Smoothing Spatial Filtering



The above is repeated for every pixel in the original image to generate the smoothed image

Smoothing an Image by Averaging



- Many possible filter parameters (size, weights, function, etc)
- **Filter size (size of neighborhood):** 3x3, 5x5, 7x7, ..., 21x21,...
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels
- **Filters function:** can be linear (a weighted summation) or nonlinear

The Filter Matrix



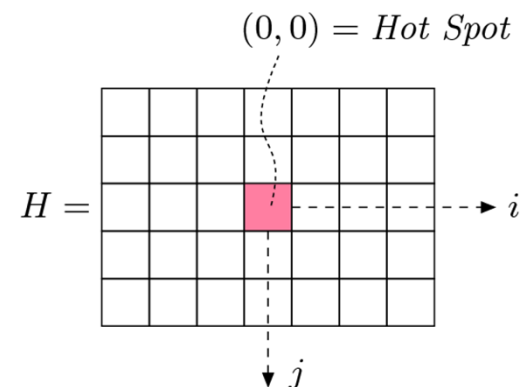
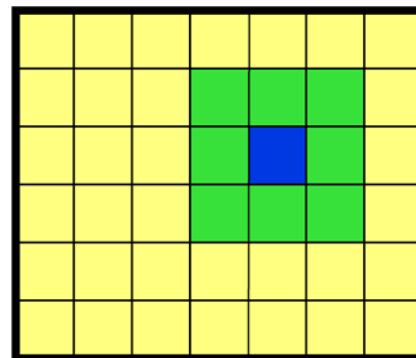
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$H(i, j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filter operation can be
expressed as a matrix
Example: averaging filter

Filter matrix also called
filter mask **$H(i, j)$**





Example: What does this Filter Do?



0	0	0
0	1	0
0	0	0



Identity function (leaves image alone)



What Does this Filter Do?



$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Mean (averages neighborhood)



Mean filter

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, img_as_ubyte
from scipy.signal import convolve2d

# Step 1: Load a built-in grayscale image
I = data.camera() # Already 2D grayscale, dtype: uint8

# Step 2: Define a 3x3 averaging filter
Window = np.ones((3, 3)) / 9

# Step 3: Apply 2D filtering (same as filter2 in MATLAB)
imfilt = convolve2d(I, Window)

# Step 4: Display the original image
plt.figure()
plt.imshow(I, cmap='gray')
plt.title("Original Image")
plt.axis('off')

# Step 5: Display the filtered image (cast to uint8 for display)
plt.figure()
plt.imshow(imfilt.astype(np.uint8), cmap='gray')
plt.title("Filtered Image (Averaging)")
plt.axis('off')

plt.show()
```



Mean Filters: Effect of Filter Size



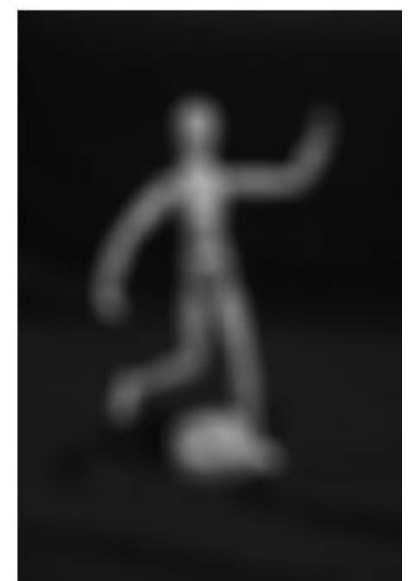
Original



7×7



15×15



41×41

Applying Linear Filters: Convolution

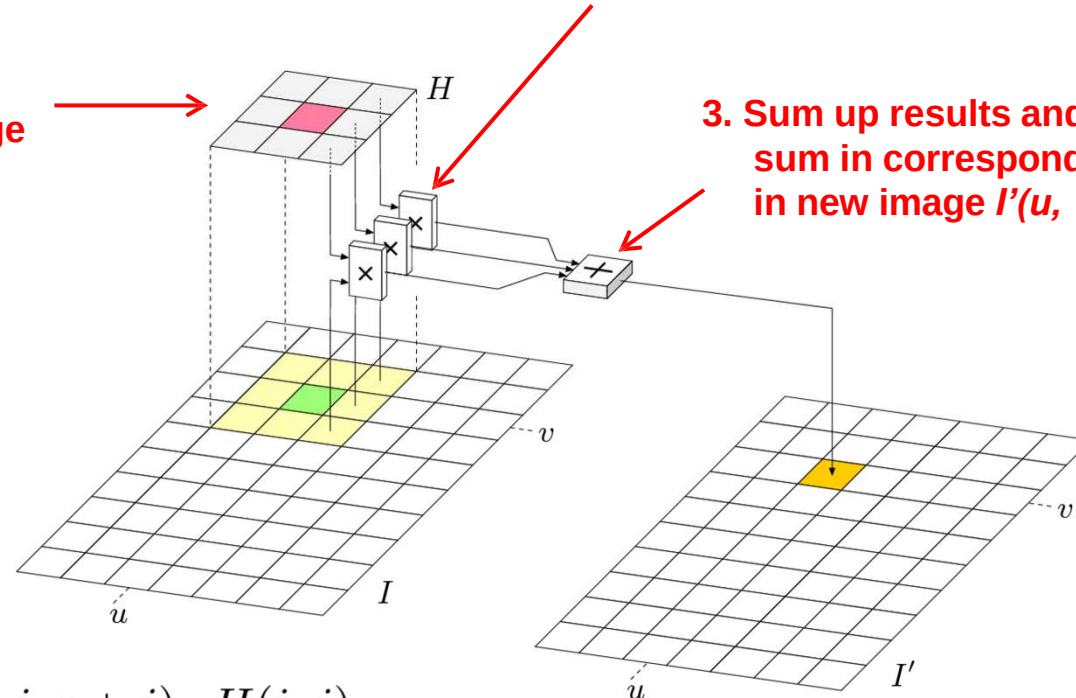


For each image position $I(u,v)$:

1. Move filter matrix H over image such that $H(0,0)$ coincides with current image position (u,v)

2. Multiply all filter coefficients $H(i,j)$ with corresponding pixel $I(u+i, v+j)$

3. Sum up results and store sum in corresponding position in new image $I'(u, v)$



Stated formally:

$$I'(u, v) \leftarrow \sum_{(i,j) \in R_H} I(u+i, v+j) \cdot H(i, j)$$

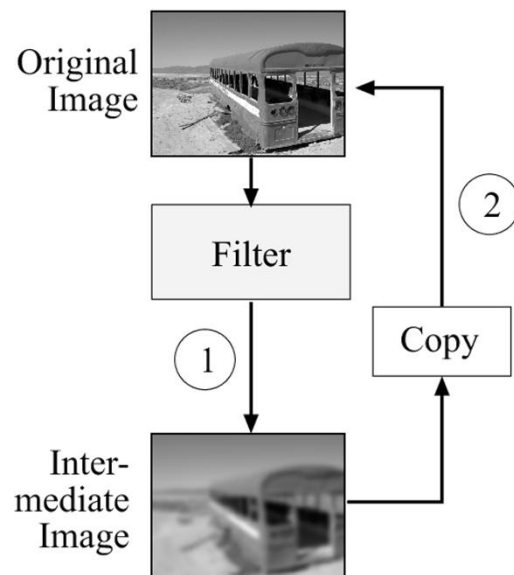
R_H is set of all pixels Covered by filter.
For 3x3 filter, this is:

$$I'(u, v) \leftarrow \sum_{i=-1}^{i=1} \sum_{j=-1}^{j=1} I(u+i, v+j) \cdot H(i, j)$$

Computing Filter Operation

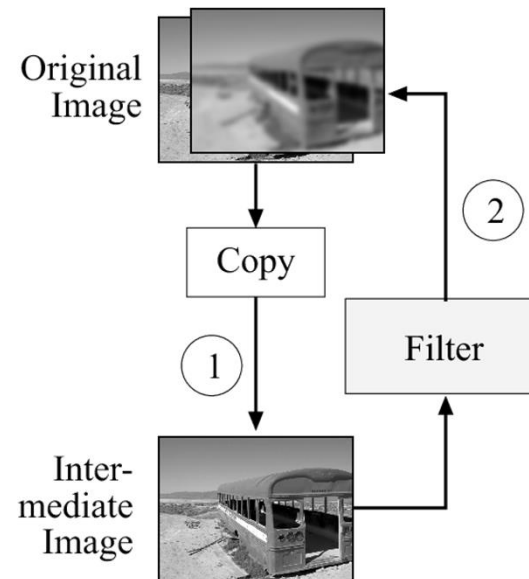


- Filter matrix H moves over each pixel in original image I to compute corresponding pixel in new image I'
- Cannot overwrite new pixel value in original image I Why?



Version A

Store results I' in intermediate image, then copy back to replace I



Version B

Copy original image I to intermediate image, use it as source, then store results I' to replace original image



Weighted Smoothing Filters

● More effective smoothing filters can be generated by allowing different pixels in the neighbourhood different weights in the averaging function

- Pixels closer to central pixel more important
- Often referred to as a *weighted averaging*

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Weighted
averaging filter



Integer Coefficients

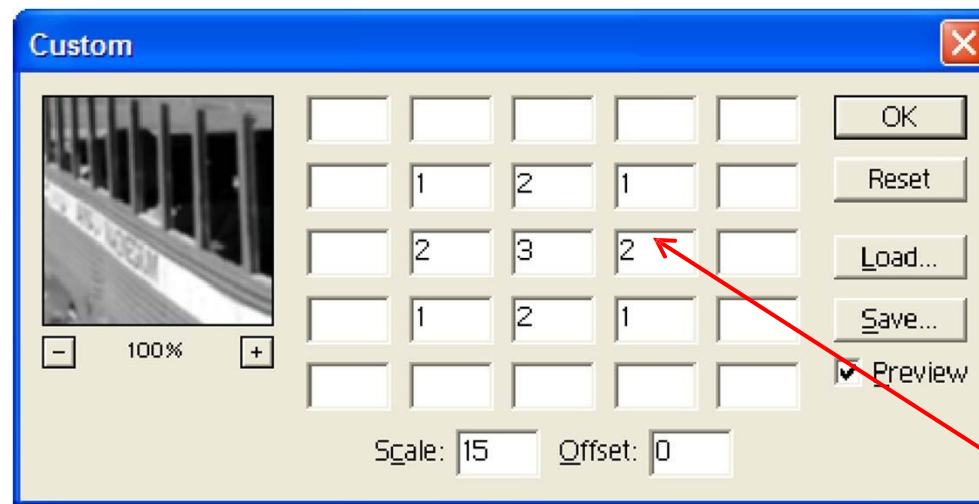
- Instead of floating point coefficients, more efficient, simpler to use:

scaling factor + integer coefficients

$$H(i, j) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & \underline{0.200} & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix} = \frac{1}{40} \begin{bmatrix} 3 & 5 & 3 \\ 5 & \underline{8} & 5 \\ 3 & 5 & 3 \end{bmatrix}$$

$$H(i, j) = s \cdot H'(i, j)$$

Example: 5x5 Filter in Adobe Photoshop



Integer filter coefficients

$$I'(u, v) \leftarrow \text{Offset} + \frac{1}{\text{Scale}} \sum_{j=-2}^{j=2} \sum_{i=-2}^{i=2} I(u+i, v+j) \cdot H(i, j)$$

If resulting pixel value is negative, offset shifts it into visible range

Scaling factor for coefficients

Median filter



```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, util
from scipy.ndimage import median_filter
```

```
# Step 1: Load grayscale image
```

```
I = data.camera()
```

```
# Step 2: Add salt & pepper noise (amount = 0.02)
```

```
J = util.random_noise(I, mode='s&p', amount=0.02)
```

```
# Step 3: Convert noisy image from float [0,1] to uint8 [0,255]
```

```
J_uint8 = (J * 255).astype(np.uint8)
```

```
# Step 4: Apply 3x3 median filter
```

```
M, N = 3, 3
```

```
K = median_filter(J_uint8, size=(M, N))
```

```
# Step 5: Display noisy and filtered images
```

```
plt.figure()
```

```
plt.imshow(J_uint8, cmap='gray')
```

```
plt.title("Noisy Image (Salt & Pepper)")
```

```
plt.axis('off')
```

```
plt.figure()
```

```
plt.imshow(K, cmap='gray')
```

```
plt.title("Filtered Image (Median Filter)")
```

```
plt.axis('off')
```



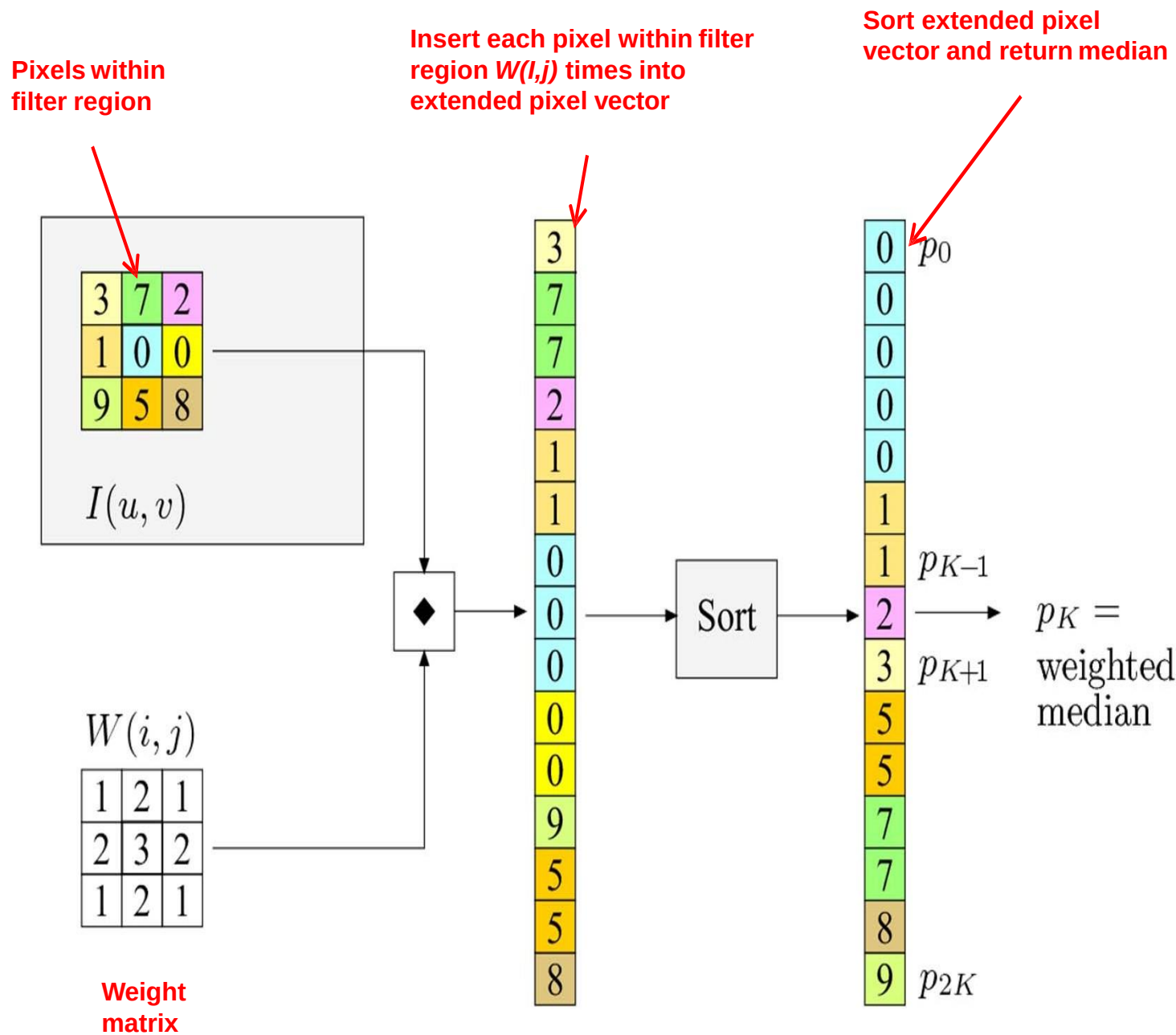
Weighted Median Filter

- Color assigned by median filter determined by colors of “the majority” of pixels within the filter region
- Considered robust since single high or low value cannot influence result (unlike linear average)
- Median filter assigns weights (number of “votes”) to filter positions

$$W(i, j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- To compute result, each pixel value within filter region is inserted $W(i, j)$ times to create **extended pixel vector**
- Extended pixel vector then sorted and median returned

Weighted Median Filter



Note: assigning weight to center pixel larger than sum of all other pixel weights inhibits any filter effect (center pixel always carries majority)!!



Weighted Median Filter

- More formally, **extended pixel vector** defined as

$$Q = (p_0, \dots, p_{L-1}) \quad \text{of length} \quad L = \sum_{(i,j) \in R} W(i,j)$$

- For example, following weight matrix yields extended pixel vector of length 15 (sum of weights)

$$W(i,j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Weighting can be applied to non-rectangular filters
- Example: *cross-shaped* median filter may have weights

$$W^+(i,j) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & \mathbf{1} & 1 \\ 0 & 1 & 0 \end{bmatrix}$$



References

- Wilhelm Burger and Mark J. Burge, Digital Image Processing, Springer, 2008
 - Histograms (Ch 4)
 - Point operations (Ch 5)
 - Filters (Ch 6)
- University of Utah, CS 4640: Image Processing Basics, Spring 2012
- Rutgers University, CS 334, Introduction to Imaging and Multimedia, Fall 2012